

Zed Language Reference Manual

Technical Manual 79-2

September 1979

**David Cheriton
Patricia Steeves**

**Department of Computer Science
University of British Columbia
Vancouver, B.C. CANADA V6T 1W5**

Copyright (c) 1979 by David Cheriton. All rights reserved.
No part of this publication may be reproduced, stored in a
retrieval system, or transmitted, in any form or by any
means, electronic, mechanical, photocopying, recording, or
otherwise, without the prior written permission of David
Cheriton.

Table of Contents

1.	Introduction	1
1.1	Zed Philosophy	1
1.2	The Zed Machine.	1
1.3	Language Structure	2
1.4	History and Development of Zed	3
1.5	Metasyntax	3
2.	Lexical Structure.	4
2.1	Comments	4
2.2	Identifiers.	4
2.3	Keywords	5
2.4	Special Characters	5
2.5	Constants.	6
2.5.1	Characters (Bytes).	6
2.5.2	Strings	6
2.5.3	Numbers	6
2.5.4	Expressions	7
2.5.5	Manifests	7
2.5.6	Templates	7
2.6	Input File Indirection	8
2.7	Conditional Compilation.	8
2.8	Relocation	9
2.9	Current Node	9
2.10	Tilde Pathnames	10
3.	Zed Programs	11

4.	Types.	12
4.1	Function Definition.	13
4.2	Data Definition.	14
4.3	Template Definition.	15
4.4	Type Conversions	16
5.	Statements	18
5.1	Data definitions	19
5.2	External Declaration	19
5.3	Template Declaration	20
5.4	Sequence	20
5.5	if	21
5.6	repeat	21
5.7	break.	21
5.8	next	22
5.9	while.	22
5.10	for	22
5.11	Word Select	23
5.12	String Select	24
5.13	goto.	24
5.14	return.	25
5.15	twit.	25
6.	Expressions.	26
7.	Operators.	27
7.1	Arithmetic	27
7.2	Bitwise.	28

7.3	Relational	28
7.4	Logical.	29
7.5	Increment/Decrement.	29
7.6	Assignment	29
7.7	Indirection.	30
7.8	Address.	30
7.9	Word Indexing.	30
7.10	Byte Indexing	31
7.11	Query	31
7.12	Function call	31
7.13	Pun	32
Appendix A	Target Machine Properties.	33
Appendix B	Common Zed Programming Bugs.	34
Appendix C	Zed Style.	35
Appendix D	Standard Zed Format.	36
Appendix E	Program Examples	38
Appendix F	ASCII Character Set.	39

Section 1: Introduction

Zed is a machine-oriented programming language in the family of BCPL, B and C, offering powerful, concise and somewhat portable constructs beyond those of assembly language, but only primitive data typing, abstraction and checking compared to many modern programming languages.

This document is a reference manual for Zed. It does not provide a tutorial introduction to Zed or programming language concepts.

1.1 Zed Philosophy

The design of Zed is based on the philosophy of providing a relatively simple, efficient-to-implement abstraction of the functionality available on many minicomputers. It is effectively a high-level assembly language because there is no run-time checking; there are only word-size data types in the language; and the language only provides features or constructs that are simple extensions of this machine. Minimal run-time support is required, making Zed suitable for low-level programming.

1.2 The Zed Machine

Zed is based on an underlying machine model which is used to define the semantics of Zed programs. Programs that make no assumptions about the underlying machine beyond this model are considered completely portable Zed programs.

The memory is a sequence of words, fixed-sized storage units consisting of at least 16 bits. Consecutive words are addressed or named by consecutive numbers. A word contains an integral number of bytes, which are each at least 8 bits. A word address can be stored in a word. Any word in memory is uniquely specified by its word address. A byte is addressed as a non-negative offset relative to a word address, the offset being the number of bytes between the first byte of the word at the word address and the desired byte.

The semantics of a Zed program follow from its execution or evaluation on this machine. There is a fairly standard set of operations on the contents of the words in memory as well as a means of accessing individual bytes. The machine uses binary 2's complement number representation and arithmetic. The result of an operation is dependent upon the types associated with the operands. The arithmetic is modulo the machine word

2 Introduction

size; overflow and underflow are ignored.

This manual describes the language as an extension of the model. Appendix A characterizes suitable hardware for implementing the language.

1.3 Language Structure

The language structure is a modest extension of the Zed machine model. Words of memory are specified by names. The name of a word is either a symbolic name associated with the address of word or an expression that evaluates to this address. A word, as a storage unit, contains a value, the contents of the word. The definition of each Zed operator includes, for each of its operands, whether it operates on the name or value of the operand.

Types are associated with operands depending on how the operands are specified. Constants such as 1, 14, 077, \$FFA8 have the default type of unsigned. Identifiers are defined with a certain type. An expression has a type derived from the last operator evaluated in the expression and the types of its operands. The definition of the operator and the values and types of each operand define the value and type of the result. Note that all types are single-word types. Zed types are:

- unsigned or non-negative integer (default type)
- integer
- null type (no operators defined on its value)
- pointer-to-T, where T is a Zed type
- function-returning-T, where T is a Zed type

These single-word types and the operators defined on them are used to implement all other data structures and operations. For example, a 1-dimensional array of n words of type T is realized by a variable of type pointer-to-T whose value is the address of the first of n words reserved for the array. Thus, arrays do not exist in the language; they are simulated with single-word types.

Typing effectively increases the number of operators without requiring new operator symbols by making an operator behave differently on operands, depending on their type. It also allows more efficient representation of values according to type in some cases. For example, pointer-to-T values are stored as the byte address of the desired word on a byte-addressed machine rather than as the word address.

A Zed program consists of one or more modules. A module is the minimal unit of separate compilation. A module is either a function definition or a data definition. Module identifiers have global scope. Modules are statically allocated and initialized in memory.

Data modules are initialized to all zeroes unless otherwise specified; space is reserved as the maximum of that specified and that required by the initialization.

Function modules are initialized with a sequence of instructions corresponding to the statements of the function; space is reserved as required for these instructions. The code is reentrant and functions can be invoked recursively. Identifiers declared within a function module have scope local to the function. Local identifiers include stack variables which are dynamically allocated for each activation of the module and labels which can only be used to transfer flow of control within the function.

Expressions are built from symbolic names and constants using operators; statements are either expressions, data definitions, data declarations, standard control flow statements or compound statements. All statements are contained in exactly one function definition. A function cannot be defined inside another function.

1.4 History and Development of Zed

Zed evolved from Eh (developed at the University of Waterloo) by adding data typing. Eh evolved from B (developed at Bell Labs) from BCPL. Compilers for Eh and Zed were implemented at Waterloo. Zed is used very little outside of Waterloo and UBC.

This document is based on information gleaned from the Eh reference materials, miscellaneous documents written at Waterloo, folklore and hearsay, and experiments with the compiler. Please notify the authors about discrepancies between this manual and the compiler(s).

1.5 Metasyntax

Boldfacing is used in the specification of syntax to indicate that the boldfaced characters are to appear literally. Non-boldfaced characters constitute meta-symbols (or non-terminals). The meta-symbol "null" is used to indicate the null string.

Section 2: Lexical Structure

Zed uses the ASCII character set with the addition of a new-line character which is the code RS (octal 036). The new-line character is intended to be used as a line delimiter/separator. The space, horizontal tab and form feed characters delimit identifiers but are otherwise ignored by the compiler. The newline character delimits comments as well as identifiers. Also, several characters have special meaning if they occur at the beginning of a line (i.e. immediately after a new-line character) as described below. The new-line character has, otherwise, no significance in the input stream of the compiler.

2.1 Comments

A comment is the text following a '\' up to the first new line character.

2.2 Identifiers

An identifier is a sequence of letters and digits; the first character must be a letter, where '.' and '_' are considered letters. Identifiers are totally case sensitive and are restricted to 32 characters in length.

The following conventions for identifiers are enforced by the compiler:

local identifiers : all letters are lower case.

globals : have at least 2 letters. The first letter is upper case, and at least one of the letters is lower case.

manifests, templates : contain at least 1 letter; no letters are lower case.

More formally,

```

<lower>      ::= a | b | c | ... | z
<upper>      ::= A | B | C | ... | Z
<numeric>    ::= 0 | 1 | 2 | ... | 9
<dot>        ::= .
<under>      ::= _

<lower+>     ::= <lower> | <numeric> | <dot> | <under>
<upper+>     ::= <upper> | <numeric> | <dot> | <under>

<local>      ::= <lower> {<lower+>}
<global>     ::= [<dot>] <upper> <lower+> {<lower+>}
<manifest>   ::= [<dot>] <upper> {<upper+>}
<template>   ::= [<dot>] <upper> {<upper+>}

```

Examples:

```

<local>      sum2
<global>     Print_results
<manifest>   .MAX_INTEGER
<template>   BINARY_TREE_NODE

```

2.3 Keywords

The following identifiers are reserved for use as keywords and may not be used otherwise:

```

active      int
break       next
case        pun
default     repeat
disable     return
else        select
enable      template
extrn       twit
for          unsigned
goto        while
if           word

```

2.4 Special Characters

The following characters have special meanings in the language:

```

operators   { } [ ] ( ) & + - * = ~ ^ | ! ? < >
escapes     * \
start of line # $ @ ? %

```

2.5 Constants

Constants and constant expressions are used in expressions and to initialize storage.

2.5.1 Characters (Bytes)

A character constant is a single letter enclosed in quotes, such as 'a'. Its value is the numeric value of the ASCII representation of the character. Certain non-graphic characters can be represented according to the following table:

*0	null character
*b	backspace
*f	form feed
*n	newline
*r	carriage return
*l	line feed
*t	horizontal tabulation
**	asterisk
**	double quote
*\$hh	hh is a 2-digit hex encoding

A single quote is written as a character constant as ''.

2.5.2 Strings

A string is a sequence of characters enclosed by double quotes, (for example "abc"). The same escape characters used for character constants are used in string constants. For example, '\"' can be used to represent '"' in a string. The value of a string constant is a pointer to the area where the characters are stored. A string is stored terminated by the null character (*0); the length of the string is not stored explicitly. The physical storage required for a string constant is the number of bytes enclosed between the double quotes plus one byte for the *0. There is no limit on the length of a string except those imposed by machine memory limitations. A real new-line character cannot appear in a string constant unless immediately preceded by '*'. In this case, the new-line is discarded. This allows the definition of multi-line string constants.

2.5.3 Numbers

A sequence of digits is considered to be an octal constant

if it begins with 0 (zero), hexadecimal if it begins with \$, and decimal otherwise. An octal representation consists of a zero followed by a sequence of octal digits. Similarly, a hexadecimal representation consists of a \$ followed by a sequence of hex digits (characters a-fA-F and digits 0-9).

2.5.4 Expressions

A constant expression is an expression in which every operand is either a constant or a constant expression. A constant expression can be used any place a constant can be used and is evaluated at compile time using 128-bit 2's complement arithmetic. This long precision means that the result of a constant expression may differ from the result if the same expression was calculated at run-time.

2.5.5 Manifests

```
manifest-def :      #identifier = text
```

A manifest definition is a line starting with a '#' in the first column. text is delimited by a new-line or '\\' (start of comment), whichever occurs first. After a manifest definition, any occurrence of the identifier as a token is replaced by text. A manifest constant may be redefined only if it is redefined to the same text as it was previously defined. It is usually best to enclose an expression in the text of a manifest with parentheses to avoid an unexpected order of evaluation after textual substitution.

Examples:

```
#ZERO    = 0
#.TRUE   = 1
#SQUARE  = (X*X)
```

2.5.6 Templates

A template defines a set of typed, named constants. The template identifier is of type unsigned and has a value 1 less than the number of words of storage described by the template. Each field of the template specified of type T is actually a constant of type pointer-to-T whose value is the pointer offset of the field from the beginning of the template. Template definitions associate data types with identifiers but do not allocate storage for these identifiers. Subsequent to their definition, templates can be used to overlay an area of

8 Lexical Structure

memory and thus associate the identifiers and data types with a storage area. Template definition is described in greater detail in the section on types.

Example:

```
template BINARY_TREE_NODE
{
    int    VALUE;
    word   LEFT[];
    word   RIGHT[];
}
```

2.6 Input File Indirection

indirection : % pathname

A line with '%' in column 1 redirects the input stream of the compiler to the specified file until a '*0' is encountered. This effectively textually substitutes the contents of the file for this line.

Example:

```
Func( word node[] )
{
    .Put_str( Father_name(node) );
    .Put_str( Brother_name(node) );
}

% /Father_name
% /Brother_name
```

2.7 Conditional Compilation

A line with '?' in column 1 is a conditional compilation statement. The line

?ifdef identifier

includes the text on subsequent lines until the next

?else

or

?end

if the identifier is defined at that point. Otherwise the text is not included in the compiler input stream and text between

```

the
    ?else
and
    ?end
is included.

```

2.8 Relocation

```
reloc-statement :    $relocate code_rbr data_rbr
```

A line starting with '\$' in column 1 is a relocation description statement. This statement can only occur at the beginning of a file and is in effect until the end of the file, except when input is redirected to another file containing another relocation statement. Relocation returns to what it was on entering the file when the end of file is reached.

The relocation statement specifies the relocation base registers (RBR's) to be used by the compiler for the code modules and data modules defined in the file. Legal values for RBR's are 1 through 7. The default values for code and data are 2 and 1 respectively.

Knowledge and explicit use of relocation is normally not required.

Examples:

```

$relocate .SYS_CODE .SYS_DATA
$relocate 1 5

```

2.9 Current Node

```
set-node-statement : @ pathname
```

A line with '@' in column 1 sets the current node of the input stream to the node specified by pathname. All subsequent relative pathnames used in input file indirection or set current node statements are relative to this current node. The current node is in effect until explicitly changed or until the end of file is reached. At the end-of-file, the current node resets to the value it had upon entering the file.

To provide for position-independence of source files within the file system, the special pathname "--me" is recognized in

10 Lexical Structure

the set-node statement. A statement of the form "@ ~me" sets the current node of the input stream to the node in which the statement occurs.

Examples:

```
@ */user/JDoe/program
@ /Main
@ ~me
```

2.10 Tilde Pathnames

A pathname occurring in an indirection or set-node statement that contains a filename component of the form "~h0" has this component replaced by the first component of the host designator string. Similarly, a component of the form "~hi" is replaced by the i-th component of the host designator string. The host designator string is specified on the command line to the compiler as "h=h0/h1..." and defaults to the machine the compiler is executing on (e.g. "ti/.10").

A further facility is provided to support the notion of a target machine. This is similar except that the filename components are specified using a 't' instead of an 'h' and the replacements are taken from the target designator string.

Examples:

```
% */src/envIRON/opcodes/~h0 \ Host machine opcodes
% */src/envIRON/crt/~t0 \ Target CRT type
@ */src/sys/kernel/~t0/~t1
```

Section 3: Zed Programs

```
program :          list-of-modules

list-of-modules :  module
                  list-of-modules module

module :          function-def
                  data-def
                  template-def
```

A Zed program is a list of modules. A module is either a function definition, a data module definition, or a template definition.

There are three levels of scope: global, global but only subsequent to definition, and local. Function and data module identifiers are global; template identifiers and template field identifiers are global subsequent to their definition; and all identifiers defined in a function are local to the function module.

Section 4: Types

A type is defined by a domain of values and the operators defined on objects in the domain. The types of Zed are:

```
unsigned (natural) numbers
int (integers)
word (null type)
pointer-to-T
function-returning-T
```

where T is a Zed type.

The three aspects of the language are: the definition of objects of these types in the language; the behavior of operators on these objects; and the initialization of storage by the definition of objects. Note: There is no user type definition facility.

```
type-def :          typed-kw ptr-funcs

type-kw :           unsigned
                    int
                    word

ptr-funcs :         ptr-funcs []
                    ptr-funcs {}
                    ptr-funcs ()
                    null

size-ptrs :         {constant}
                    [constant] size-ptrs
                    ptr-funcs
                    null

typed-ident0 :      identifier
                    type-kw identifier

typed-ident1 :      type-kw identifier ptr-funcs
                    identifier ptr-funcs

typed-ident2 :      type-kw identifier size-ptrs
                    identifier size-ptrs

typed-ident3 :      identifier size-ptrs
                    identifier ptr-funcs

typed-ident4 :      identifier ptr-funcs
```

A type is specified by giving the type as an extension of the base type. The base type is specified by one of the key words: unsigned, int or word; absence of an explicit base type causes the base to default to unsigned. An extension of the

base type is specified by suffixing special symbols. Appending '[' or '{' to a specification of type T makes it of type pointer-to-T. Appending '()' to a specification of type T makes it of type function-returning-T. The position of these suffixes depends on where the type specification is used. The lack of uniformity in type specification prompts elaboration as each case is discussed. The definition of objects is divided into function objects and data objects.

typed-ident0's are used in function definitions.
 typed-ident1's are used in function parameter declarations.
 typed-ident2's are used in external definitions.
 typed-ident3's are used in local data definitions.
 typed-ident4's are used in external declarations.

Examples:

```
typed-ident0 :      int Func
typed-ident1 :      word node[]
typed-ident2 :      Header{15}
typed-ident3 :      value
typed-ident4 :      Header{}
```

4.1 Function Definition

```
function :          typed-ident0( params ) ptr-funcs stmt
params :           null
                  ?
                  formal-param-list
                  ; formal-param-list
                  formal-param-list; formal-param-list
.
formal-param-list : typed-ident1
                  formal-param-list, typed-ident1
```

A function identifier is defined to be of type function-returning-T where T is specified by the typed keyword and type suffixes, if any. The identifier is the name of the first word of storage allocated for the module. This storage is initialized with the machine code corresponding to the statement.

The identifiers in the formal parameter list are defined only over the scope of the statement. Note: all parameters must be typed separately and default to unsigned if no type is specified.

Parameters following a ';' in the formal parameter list definition are optional and need not be assigned in a function call. The function .Nargs() returns the number of actual arguments passed to the function. A formal parameter list specified by '?' is used in functions that can receive an

14 Types

arbitrary number of actual arguments. The function .Arg(n) is used in these functions to return the n-th actual argument.

Besides formal parameters, other data objects can be defined inside a function definition. All such objects, including the parameters, only exist during the evaluation of the function; there are separate instantiations of these objects for each evaluation; and the scope of the identifiers for these data objects is local to the function. Therefore, all parameters are passed by value.

The only valid operators on an object of type function are the function call operator and the address operator.

Examples:

```
Main()  
Abs( int a ) return( .Abs(a) );  
int Compute( ? )  
Concatenate( str1{}, str2{}; str3{} ){}[]
```

4.2 Data Definition

```
data-definition :    typed-ident2;  
                   typed-ident2: constant;  
                   typed-ident2: initial-list;  
                   typed-ident2: string;  
  
initial-list :      initial-element  
                   initial-list, initial-element  
  
initial-element :    {initial-list}  
                   constant  
                   identifier  
                   string
```

A data identifier is defined with the specified type and with global scope. Storage is allocated and initialized. One word is allocated for the cell specified by the identifier. Further storage is allocated as the maximum of that required by the initial list or string and that specified by the size. Storage that is not explicitly initialized is initialized to all zeroes. Explicit initial values are:

constant	its value.
string	a pointer to storage that is allocated and initialized with the string.
identifier	a pointer to the word named by the identifier. The identifier must be defined with global scope.

[initial-list] a pointer to an area where the elements of initial-list have been stored consecutively.

For a definition of the form

```
identifier{size}: initial-string
```

an array is allocated of (size+1) bytes or the number of bytes required by the initial string, whichever is larger. Similarly, for a definition of the form

```
identifier[size]: initial-list
```

an array is allocated of (size+1) words or the amount required by the initial-list, whichever is larger. In both cases identifier is initialized to point to the area allocated.

Examples:

```
word Node[];
Verbose: .FALSE;
int Array[2]: -1, 0, 1;
unsigned Names[0]{}: "Jim", "Peter";
Str{80}: "abc";
```

4.3 Template Definition

```
template-def :      template identifier{ field-list }

field-list :        field-group
                    field-list; field-group

field-group :        type-kw. field
                    field-group, field

field :              typed-ident1
                    typed-ident1:{constant}
                    typed-ident1:[constant]
```

A template defines a set of typed, named constants. The template identifier is of type unsigned and has a value 1 less than the number of words of storage covered by the template. Each field of the template specified of type T is actually a constant of type pointer-to-T whose value is the pointer offset of the field from the beginning of the template. Template definitions associate data types with identifiers but do not allocate storage for these identifiers. Subsequent to their definition, templates can be used to associate the identifiers and data types with a particular storage area.

A template is used to associate a heterogeneous sequence of types with a block of storage. This is similar to the DSECT

16 Types

facility of the OS/360 assembler and the record facility in AlgolW. A field name of the form

`typed-ident1:{constant}`

defines this field to be `constant+1` bytes long with the identifier having as value the offset of the beginning of the field within the template. A field name of the form

`typed-ident1:[constant]`

is the same except the length is `constant+1` words instead of bytes.

Examples:

```
template NODE{ unsigned VALUE ; word L[], C[], R[]; }
```

```
template EMPLOYEE
{
    unsigned    NAME{};
    unsigned    PAYCHECK[:[11]]; \ First of 12 words
}
```

4.4 Type Conversions

The types unsigned, int, pointer-to-word and pointer-to-T form a hierarchy with unsigned the lowest (default) type and pointer-to-T being the highest. Implicit promotions occur in the type hierarchy as required.

`unsigned --> int --> pointer-to-word --> pointer-to-T`

These implicit type conversions on objects can change the value of the object. For example, on a byte-addressed machine, promotion of an unsigned object to pointer-to-T doubles the value to produce the corresponding byte address if pointers are stored as byte addresses and there are two bytes per word. When a value `i` of type unsigned or int is promoted to pointer-to-word, it then points to the `i+1`st machine word in the logical address, counting 0 as the 1st word.

There is no implicit conversion of objects to or from type function-returning-T or objects of type word. There is also no implicit conversion of objects to lower types.

Explicit conversions are possible using the pun operator.

```
name :                pun( type-def, name )
value :               pun( type-def, value )
```

The result type is the type specified. The value is the bit pattern of the value of the second argument interpreted as the resultant type. This operator can introduce implementation dependencies; its use should be minimized as a matter of style and portability.

Section 5: Statements

```

stmt :          simple-stmt;
              label: stmt
              compound-stmt
              ;

simple-stmt :    expression
              data-def-stmt
              extrn-dcl-stmt
              template-dcl-stmt
              transfer-stmt
              twit-stmt

transfer-stmt : break-stmt
              next-stmt
              goto-stmt
              return-stmt

compound-stmt : seq-stmt
              if-stmt
              repeat-stmt
              for-stmt
              while-stmt
              word-select
              string-select

```

A statement can only occur as the statement in the definition of a function or else as one of the statements in a statement construct. Zed provides the standard control flow constructs that compose a set of statements into a particular pattern of execution (or flow) to form a statement. There is also a data definition statement for defining local data objects in a function; an external declaration statement that declares the use of a global identifier within the function; and a template declaration statement that declares the use of the template name and its field names within the function.

Note that the ';' is a statement terminator rather than a statement separator as in ALGOL-like languages. Also, a statement label is an identifier that is defined by its occurrence preceding a statement.

In the following, the data definition statements and the external and template declaration statements are referred to as non-executable; all other types of statements are executable. Non-executable statements must precede all executable statements in a function. Labels can occur on executable statements only.

5.1 Data definitions

```

data-def-stmt :      type-kw typed-ident-list

typed-ident-list :   typed-ident3
                    typed-ident-list, typed-ident3

```

The data definition statement defines the specified identifiers local to the function. These identifiers name variables that exist only during the invocation of the function and are distinct for each invocation of the function. Data definition statements must precede all executable statements.

The value of an identifier of the form:

```
identifier[size]
```

is a pointer to an array of (size+1) words that is allocated on the stack at function entry. Similarly, the value of an identifier of the form:

```
identifier{size}
```

is a pointer to an area of (size+1) bytes. If size is null, however, no space is allocated other than one word required by the pointer itself and this word is not initialized.

Examples:

```

unsigned i, j, str{}, name[] {20};
int list[100], factor;
word dummy, data[];

```

5.2 External Declaration

```

extrn-dcl-stmt :      extrn typed-extrn-list

typed-extrn-list :    extrn-list
                    type-kw extrn-list

extrn-list :          typed-ident4
                    extrn-list, typed-ident4

```

The external declaration statement declares the use of the specified globally defined identifiers within the function. Their type declaration must agree with their definition. External declaration statements must precede executable statements. All global identifiers used in the function must be declared, except for function names used as the name in a function call.

Examples:

```
extrn Page_no, Line_no, Border[]{};
extrn int Format_type;
extrn word Index_file[];
```

5.3 Template Declaration

```
template-dcl-stmt :  template temp-list

temp-list :          identifier
                  temp-list, identifier
```

The template declaration statement declares the use of the specified templates within the function. Each identifier must be the name of a template. A template must be declared in the function before its name or its field names are defined. Template declaration statements must precede executable statements.

The definition of the template must precede the definition of any function that uses this template. Two templates used in the same function having a common field name must define this field to be of the same type and value.

Examples:

```
template NODE;
template BRANCH, EMPLOYEE, CUSTOMER;
```

5.4 Sequence

```
seq-stmt :          { stmt-list }

stmt-list :          stmt
                  stmt-list stmt
```

The statements in the statement-list are executed sequentially.

Examples:

```
{ .Put( c ); ++i; }

{
  x = integer[y];
  if( x < 0 ) x = -x;  \ Absolute value
}
```

5.5 if

```

if-stmt :           if( expr ) stmt
                  if( expr ) stmt else stmt

```

If `expr` is non-zero, the first statement (the if clause) is executed. Otherwise, the second statement (the else clause), if any, is executed. An else clause is associated with the first immediately preceding if clause that does not have an else clause.

Examples:

```

if( Odd(value) ) ; \ Do nothing

```

```

if( i == limit )
    return( sum );
else
{
    sum += value;
    ++i;
}

```

5.6 repeat

```

repeat-stmt :      repeat stmt

```

`stmt` is repeatedly executed until the loop is terminated by a `break` or `return` statement.

Example: find length of a string.

```

i=0;
repeat
{
    if( str[i] == '*0' ) break;
    ++i;
}

```

5.7 break

```

break-stmt :       break

```

`break` terminates the execution of the immediately enclosing loop in the function.

5.8 next

next-stmt : next

next transfers control to the beginning of the immediately enclosing loop in the function.

Example: iterate on non-digits only.

```

i=0;
while( c=str[i++] )
{
    if( c>='0' && c<='9' ) next;
    ++num_letters;
}

```

5.9 while

while-stmt : while(value) stmt

The while-statement is equivalent to:

```
repeat{ if( !value ) break; statement }
```

Example: output a list in reverse order.

```

i = num_items;
while( i > 0 )
{
    --i;
    .Put_str( list[i] );
}

```

5.10 for

for-stmt : for(expr-1; expr-2; expr-3) stmt

expr-1 : value
 null

expr-2 : value

expr-3 : value
 null

The for-statement is equivalent to:

```

expr-1;
while( expr-2 )
{

```

```

        statement
        expr-3;
    }

```

Example: print 80 asterisks.

```

    for( i=0; i < 80; ++i) .Put('**');

```

5.11 Word Select

```

word-select :      select( value ) { word-case-list }

word-case-list :   word-case
                   word-case-list word-case

word-case :        w-label : stmt
                   w-label : word-case

w-label :          case constant
                   case relational-op constant
                   case constant1 :: constant2
                   default

                   ( "::" means "through")

```

The word select statement is equivalent to a chain of if-else statements in which the test expression is one of the following.

```

( value == constant )
( value relational-op constant )
( constant1 <= value <= constant2 )

```

The default case corresponds to the final else in the chain. If no case is matched and there is no default case, execution continues at the next statement. Case ranges must be mutually exclusive.

Example: underline alphanumeric characters only.

```

select(c)
{
    case '0'::'9':
    case 'a'::'z':
    case 'A'::'Z':
        Underline(c);
}

```

5.12 String Select

```

string-select :      select{ value } { str-case-list }
str-case-list :      str-case
                     str-case-list str-case

str-case :           s-label : stmt
                     s-label : str-case

s-label :             case string-constant
                     default

```

The string select statement is logically equivalent to a chain of if-else statements. value must be type pointer-to-unsigned and point to a string; it is compared with the string constants of each case. If a match is found, the associated statement is executed. Otherwise, the default statement (if any) is executed. Each of the string constants must be unique.

Examples:

```

select{s}
{
    case "+a":
        A = .TRUE;
    case "-b":
        B = .FALSE;
    default:
        .Put_str("Error: Invalid option*n");
}

```

5.13 goto

```
goto-stmt :      goto label
```

Control is transferred to the statement prefixed by "label:". The label must be defined within the function.

Example: print 10 blank lines

```

        i=1;
loop:   if( i > 10 ) goto quit;
        .Put('*n');
        ++i;
        goto loop;
quit:   .
        .
        .

```

5.14 return

```
return-stmt :      return  
                return( value )
```

Execution is resumed at the point where the function was invoked. The return of a value effectively assigns this value to the name of the function and therefore must satisfy the type-compatibility restrictions of assignment.

5.15 twit

```
twit-stmt :      twit( param_list )
```

The twit statement is used to insert machine (or native) code directly into a program. The form and number of its parameters and its semantics are machine dependent. Because this is of little merit outside of the lowest-level system software, a description of the twit statements for various machines is included in the compiler manual only.

Section 6: Expressions

```

expression :      primary
                  expression binary-op expression
                  expression ? expression : expression

primary :         constant
                  identifier
                  preuni-op primary
                  primary postuni-op
                  ( expression )

```

An expression is either a simple constant, an identifier, or an operator whose operands are expressions. The result of evaluating the expression is the value of the constant or identifier or the result of the operator. The type of the result is the type of the constant or identifier or else the type of the result of the last operator executed.

Each operator has an associated precedence that affects when it is evaluated in an expression. Operators are listed below in decreasing precedence.

```

0   ()
1   ++, -- (postfix), the array operators, function calls
2   ++, -- (prefix), -, ~, *, !
3   <<, >>, &, *, !, ^, |
4   *, /, % (binary)
5   +, - (binary)
6   ==, !=, <, <=, >, >=
7   &&
8   ||
9   ?
10  all assignment operators

```

The order of execution of operators to evaluate an expression in Zed is well-defined. The order is determined by the precedence and order of the operators and the order of evaluation of the operands for each operator.

In the evaluation of an expression, the expression is parsed into one of the above decompositions such that the operator in the decomposition has the lowest precedence in the expression. If there is more than one operator of this precedence, the rightmost operator is chosen, except for assignment, array and query operators in which cases the leftmost operator is chosen. The operands of the chosen operator are evaluated in the order defined for this operator and then the operator is evaluated on the results from its operands. The evaluation of the operands involves evaluating subexpressions which follows these same rules. The parenthesis operator serves only to explicitly control the order of expression evaluation.

Section 7: Operators

The definition of an operator includes: the order of evaluation of its operands; whether it operates on names or values; valid types for the operands; and the type and value of the result as a function of the operands.

The order of operand evaluation is well-defined. Except for the assignment, indexing and function call operators, operands are evaluated left to right. With the function call operator, the arguments are evaluated left to right, the function name is evaluated, and then the function call is performed.

Most operators use the value of their operands. If a name is given as an operand where a value is required, the value of the word specified by the name is used. It is an error to give a value where a name is required.

Operands of type included in the type hierarchy are promoted as required by the operator for compatibility with the other operands.

unsigned --> int --> pointer-to-word --> pointer-to-T

Valid types and the type of the result are described for each operator. The type "function-returning-T" is only valid as an operand for the function call and address operators; type word is only valid as the operand of the address operator. In the following, all other types for operands are valid unless otherwise stated.

7.1 Arithmetic

value : value binary-arith-op value
 unary-arith-op value

binary-arith op : +,-,*,/,%

unary-arith-op : -

+ algebraic sum
- (binary) difference
* product
/ quotient
% remainder of the division

The remainder is defined such that given x and y, the following relationship holds:

$$x = (x \% y) + (x / y) * y$$

- (unary) negation

The arithmetic is modulo the machine word size; overflow and underflow are ignored. The type of the result is the maximum (in the type hierarchy) of the types of its operands. For example, if the operands to + are unsigned and pointer-to-T, the result is pointer-to-T. If both operands of + or - are pointers, either one must be pointer-to-word or both must be pointer-to-T, for some T. Except for + and -, operands must be type unsigned or int.

7.2 Bitwise

value : value bitwise-op value
 ~ value

bitwise-op : <<, >>, &, ^, |

~ logical one's complement
<< shifts left operand left by the number of
 bits specified by the right operand
>> is similar to << but shifts to the right
& bitwise and
^ bitwise exclusive-or
| bitwise or

Operands are treated as strings of bits one word in length. Vacated bit positions in the result of a shift operation are zero-filled. Operands must be type unsigned or int; result is type unsigned.

7.3 Relational

value : value relational-op value

relational-op : ==, !=, <, <=, >=, >

== equality
!= inequality
< arithmetic less than
> arithmetic greater than
<= arithmetic less than or equal to
>= arithmetic greater than or equal to

Operands are promoted as required to comparable types. The

result is type unsigned: 1 if the relation is true; 0 if the relation is false.

7.4 Logical

value : value binary-logical-op value
 unary-logical-op value

binary-logical-op : &&,||

unary-logical-op : !

&& logical and
|| logical or
! logical complement

The value of each operand is true if non-zero, false if zero. The result is type unsigned: 1 if true; 0 if false. The second operand of '&&' is not evaluated if the first operand evaluates to 0. The second operand of '||' is not evaluated if the first operand evaluates to non-zero.

7.5 Increment/Decrement

value : name inc-dec-op
 inc-dec-op name

inc-dec-op : ++,--

The value of name is incremented (decremented) by 1. If the operator precedes the operand, the result is the value of the operand after it has been incremented (decremented). If the operator follows the operand, the result is the value of operand before it is incremented (decremented). The type of the result is the type of the operand. Incrementing (decrementing) an operand of type pointer causes it to point to the next (previous) word.

7.6 Assignment

value : name assign-oper value
 byte-name = value

assign-oper : =, +=, -=, *=, /=, %=, >>=, <<=, &=, ^=, |=

'=' is the conventional assignment operator; value is stored in the word named by name (or byte in the case of byte-name). The type of the right operand must be such that it can be promoted to the type of the left operand unless both operands are pointers. The type of the result is the type of the left operand. Expressions using the other assignment operators are equivalent both in effect and type restrictions to:

name = name x value

where x is the operator symbol that precedes '='. Name is only evaluated once.

7.7 Indirection

name : *value

name is the name of the word whose address is specified by value. The operand must be type pointer-to-T; the result is type T.

7.8 Address

value : &name

value is a pointer to the word specified by name. If the operand is type T, the result is type pointer-to-T. The operand may be of type word or function-returning-T in addition to unsigned, int, and pointer-to-T.

7.9 Word Indexing

name : value1[value2]

Normally, value1 is type pointer-to-T and value2 is type unsigned. name is then the n-th word after the word pointed to by value1, where the 0-th word is the word pointed to by value1. In this case, the type of the result is T. Except for side effects in the operand evaluation,

e1[e2] == e2[e1] == *((e1) + (e2)) == *((e2) + (e1))

where '=' denotes equivalence and e1, e2 are expressions.
 Note: by this equivalence, at least one of value1 and value2 must be of type pointer-to-T.

7.10 Byte Indexing

byte-name : value1{ value2 }

byte-name names the byte value2 bytes offset from the word specified by value1. The left operand must be type pointer-to-unsigned, the right operand must be unsigned or int. The result is type unsigned: the value is the contents of the byte right-justified and zero-padded.

Note: Byte-name can only be used as a name as the left operand of a simple assignment statement; it can be used as a value anywhere an unsigned value can be used.

7.11 Query

value : value-test ? value-T : value-F

The leftmost operand is evaluated first; if it is non-zero, the result is the value of the second operand, otherwise the value of the third operand. Only one of the second and third operands is evaluated. The type of the result is the maximum of the types of the last two operands.

7.12 Function call

value : name (arg-list)
 arg-list : null
 non-null-arg-list
 non-null-arg-list : value
 non-null-arg-list, value

The operands in the argument list are evaluated left-to-right, the first operand (function name) is evaluated, and then the function is evaluated. name must evaluate to type function-returning-T; the result is type T. The value of the result is the value assigned to name by a return statement in the function if any; otherwise the value is undefined.

Evaluation of the function consists of, for each argument, assigning its value to the corresponding formal parameter of the function and evaluating the statement specified in the function definition. The types of the arguments must satisfy the type compatibility restrictions of assignment with respect to the formal parameters. All arguments are passed by value.

If the function is defined with n formal parameters but the last k are optional, the argument list may contain m arguments, for any m in the range $n-k \leq m \leq n$.

7.13 Pun

name :	pun(type-def, name)
value :	pun(type-def, value)

The result type is the type specified. The value is the bit pattern of the value of the second argument interpreted as the resultant type. This operator can introduce implementation and machine dependencies; its use should be minimized as a matter of style and portability.

Appendix A: Target Machine Properties

33

Zed is normally compiled into the native or machine code of the target machine hardware (the machine that executes the compiled code). For this generated code to be efficient in space and time, the hardware must satisfy certain requirements.

Zed is implemented with a stack which is used to allocate stack frames, one per function invocation. A stack frame is used to store function arguments, local variables and function linkage information so an efficient means of implementing a stack is required. Moreover, locations in a stack frame may be referenced in an arbitrary order. A dedicated index register usually suffices for stack implementation. The hardware stack support found on a number of minicomputers has not been useful for implementing the Zed stack.

The abstraction of the Zed machine with a word memory should be efficient to implement.

Further assumptions are made about the host hardware by the so-called machine-independent parts of the Zed compiler.

Appendix B: Common Zed Programming Bugs

1. Failure to initialize variables.
2. Misuse of the special assignment operators.
3. Use of the assignment '=' instead of '==', especially in if statements.
4. Misunderstanding of order of evaluation of an expression.
For example,

```
if ( i = j && k )
```

is equivalent to

```
if ( i = ( j && k ) )
```

5. Confusion over post- and pre-increment/decrement.
6. Testing for greater or equal to zero with unsigned numbers. The comparison is always true.

Appendix C: Zed Style

Zed has a potentially destructive level of flexibility; discipline of style is required on the part of the programmer to avoid unmaintainable programs. The following principles are prudent to follow:

1. Functions should be less than a page long. Many small functions or few large functions make reading and understanding a program difficult.
2. The use of global data should be kept to a minimum. Ideally, global data should not be modified or only modified by one function.
3. More than 4 or 5 parameters suggests a poor function definition. However, communication with functions should be largely via argument-passing.
4. The use of twits and puns should be avoided, or at least critically appraised when used.
5. Limited use of the goto statement seems beneficial; its use is rarely justified.
6. The complex expressions possible in Zed with increment, decrement and nested assignments are a common source of bugs. A sequence of simpler statements is frequently more readable and more likely to be correct.
7. The Zed for statement provides for the execution of four statements in a strange order. Its use is best limited to constructs similar to the for statement in Pascal.
8. Zed supports reasonable naming of constants through manifests and templates. Judicious use of this facility can greatly improve the portability and readability of programs.
9. Use of pointers, pointer manipulation, and the unary operators * and & should be minimized. Pointers are best used logically as pointers to arrays, a sequence of words of the same type, or records, a sequence of words of different types. Arrays are accessed by indexing, and records are accessed using a template and a word pointer to the beginning of the record.
10. Any doubt about the order of evaluation of an expression should be removed by parenthesis. This is a common difficult problem.

Beyond this, the standard principles of program design, structured programming and documentation should be applied.

Appendix D: Standard Zed Format

There is a standard format for writing Zed program source code that is followed in most Zed programs. These conventions are intended to improve readability through uniformity and generally good layout.

Most programs are stored in a tree of files with one function per file named the same as the function, a file of manifests containing the manifest constants used, a file called externals for the external data module declarations, a file for each template used with the name of the file being the name of the template, or else a file called templates. The root of the tree of source is a file of indirections to the files in the subtree. Normally only source is stored in a source subtree.

Lines of source are usually no more than 72 characters long so they will fit on a standard CRT screen as well as on eight and one-half by eleven inch paper with an offset from the left margin to allow insertion in a binder.

Lines are indented from the left margin using spaces and tabs with tab stops defined every 4 columns. The first line of the declaration of a function, global data module, template or manifest is not indented. Opening brackets '{' are indented 2 spaces from the current indentation and move the current indentation right one tab (which is equivalent to 4 spaces). Closing brackets '}' are indented 2 spaces less than the current indentation and move the current indentation left one tab (i.e. one fewer tab is used). Brackets are placed on separate lines from the statements they enclose.

Comments are horizontally aligned with what they describe on a separate line. Otherwise, they are separated by at least one tab from the end of the program source. An attempt is made to align comments that are appended to source lines.

Blank lines are used to space out source lines, effectively grouping logical units of statements. Similarly, spaces are used to make expressions and statements clearer. Normally, a binary operator is separated from each of its operands by a space and parentheses are separated from what they enclose by a space at each end. In complex expressions, expressions first loose parenthesis-spaces and then operator-spaces with increased nesting. Spaces are not used to separate if from the following left parenthesis, similarly for while, select, return, twit, for and function identifiers. Spaces are also not used before the word- and byte-indexing operators.

Identifiers follow several conventions. Global names (such as functions and data modules) have at least two letters. The first letter is upper case and at least one letter is lower case. All names with local scope are lower case. Names of

manifests, templates and fields of templates are all upper case. These conventions are enforced by the compiler.

Data modules are formatted as follows. The name starts a new line with initialization on the same line if it will fit. If the initialization contains multiple fields that require comments or are not simple constants themselves, each field is put on a separate line. In this case, the enclosing square brackets are put on separate lines, indented as described for the curly brackets above. For example:

```
Descriptor [10][]:
    [
        "tty",                \ name
        TTY_INPUT,            \ input index
        [                      \ sub-descriptor
            $4C00,             \ interface address
            0,
        ],
        .Open_tty,            \ open device function
    ];
```

A function module is formatted as follows: The name and formal argument list occurs on the first line followed by a blank line, followed by declarations, each starting a new line indented one tab. Declarations are ordered templates, external references, local declarations. The latter are ordered internally by word, int, unsigned. The declarations are separated from the rest of the function body by a blank line. The degree of indentation changes with the bracketing. The last closing bracket is indented two spaces. For example:

```
Main( optc, argc, cmd_vec[]{} )
    \ Print options and arguments to standard output
    {
        unsigned i;

        for( i=0; i < optc+argc; ++i )
            .Printf( "%s*n", cmd_vec[i] );

        .Close( .Select_output() );
    }
```

Appendix E: Program Examples

The following program examples are included to illustrate aspects of the language.

Return the maximum of two unsigned numbers

```
.Max( a, b ) return( a > b ? a : b );
```

Shift left the character string in the first argument by the number of characters specified in the second.

```
.Shift_left( str{}, chars ){}
{
    unsigned    i;

    i = 0;
    while( str[i] = str[i+chars] ) ++i;

    return( str );
}
```

Concatenate the strings given as arguments. This illustrates the use of ? in the formal parameter list, the built-in functions .Args and .Nargs, and the use of the pun operator.

```
.Concat( ? ){}

\    call: .Concat( dest, s1, s2, ... )
\    The strings s1, s2, ..., are concatenated and the
\    result stored in dest which is the returned value.
\    dest is assumed to have enough room to store the
\    resulting string.
{
    unsigned    dest{}, str{};
    unsigned    i, dest_pos, j, nargs;

    nargs = .Nargs();
    dest = pun( unsigned{}, .Arg(1) );
    dest_pos = 0;

    for( i = 2; i <= nargs; ++i )
    {
        str = pun( unsigned{}, .Arg(i) ); \ i-1st source string

        j = 0;
        while( dest{dest_pos} = str{j++} ) ++dest_pos;
    }
    return( dest );
}
```

Appendix F: ASCII Character Set

X Y \	0	1	2	3	4	5	6	7
0	nul dle			0	@	P	`	p
1	soh dcl	!	!	1	A	Q	a	q
2	stx dc2	"	"	2	B	R	b	r
3	etx dc3	#	#	3	C	S	c	s
4	eot dc4	\$	\$	4	D	T	d	t
5	enq nak	%	%	5	E	U	e	u
6	ack syn	&	&	6	F	V	f	v
7	bel etb	'	'	7	G	W	g	w
8	bs can	(	(	8	H	X	h	x
9	ht em	)	)	9	I	Y	i	y
10	lf sub	*	*	:	J	Z	j	z
11	vt esc	+	+	;	K	[	k	{
12	ff fs	,	,	<	L	\	l	
13	cr gs	-	-	=	M	]	m	}
14	so rs	.	.	>	N	^	n	~
15	si us	/	/	?	0		o	del

The numeric value of a character is calculated from its coordinates in the table as

$$\text{value} = 16 * X + Y$$

This is equivalent to taking y as the low-order 4 bits and x as the high-order 3 bits composing the 7-bit ASCII code.

